

ТЕМА 2.
**ТРАНСЛАЦИЯ, ПРОЦЕС НА ПРОГРАМИРАНЕ,
ГРЕШКИ, СРЕДИ ЗА ПРОГРАМИРАНЕ**

УВОД

- Всеки може да създаде нов език, респ. нов ЕП.
- Необходимо е обаче новият език да бъде прилаган на практика за комуникация, респ. за програмиране.
- След като вече знаем някой съществуващ (използваме го до момента), за да прилагаме и новия ни е необходимо средство за превод.
- Всеки новосъздаден ЕП трябва да може да бъде „изпълняван“ на компютър. Компютърът „разбира“ машинен език.
- Т.е. е необходимо да бъде създаден т.нар. **транслатор**, нар. още **езиков процесор**

ПРОЦЕСЪТ НА ТРАНСЛАЦИЯ

За да бъде **създаден** ЕП от високо ниво той трябва да се **реализира** чрез някой от преди това съществуващите и реализирани ЕП т.е. в крайна сметка той трябва да бъде **реализиран на машинния език**, който е „разбираем“ за хардуера, а не чрез софтуер.

За да бъдат изпълнявани програми на един ЕП от високо ниво те трябва да могат да се превеждат (translate) на машинен език т.е. написаните от програмиста програми в текстов вид (source code) трябва да се преведат във вид, който може да се изпълни от компютъра (byte, executable code)

- **Транслация (translation)** – процесът на превеждане на текстовия код на програмата в изпълним (двоичен) код
- **Транслатор (translator)** е инструментът, който превежда текстовия код на програмата в изпълним код

КОМПИЛАТОРИ И ИНТЕРПРЕТАТОРИ

- Видове транслятори (ЕП обикновено се реализира чрез):
 - **Компилятор** (*compiler*) – превежда еднократно цялата програма от текстовия код на езика (*source language*) в програма на друг ЕП (обектен език, *object language*)
 - **Интерпретатор** (*interpreter*) – чете кода на програмата от езика и я изпълнява директно твърдение след твърдение, интерпретатора също е зареден в ОП
- Сравнение на компилаторите и интерпретаторите:

Показатели	Компилатори	Интерпретатори
<i>Реализация</i>	по-трудно	по-лесно
<i>Време за компилиране на програмата</i>	повече	по-малко
<i>Време за изпълнение на програмата</i>	по-малко	повече
<i>Многократно изпълнение на програмата</i>	по-ефективни при многократно изпълнение	по-бързи при еднократно изпълнение
<i>Вид на разработвания софтуер (най-често)</i>	за широко използван софтуер (C++, Java, C#)	за специализиран софтуер (SQL, Lisp, Prolog)

ДРУГИ ЕЗИКОВИ ПРОЦЕСОРИ

Асемблери- транслират програмата от мнемоничен в машинен код.

Пример за мнемоничен код

MOV AL, 61h ; Зарежда регистъра AL с десетично 97
(шестнадесетично 61)

Дисасемблери- транслират от машинен код в мнемоничен код.

Конвертори- превеждат от език от високо ниво на друг език от високо ниво.

Емулатори- дават възможност програма на машинен език за една микропроцесорна система да бъде изпълнена на друга микропроцесорна система.

РАЗРАБОТКА НА СОФТУЕР – ЖИЗНЕН ЦИКЪЛ

- Какво е приложение (нар. още информационна система)? Обикновено е множество от компютърни програми или нар. компоненти. Например MS OFFICE 2016 съдържа 31 изпълними файла само в кореновата папка. Един от тях е MSOUC.EXE (Microsoft Office Upload Center) – използва се от всички за ъплоад на файлове.
- Основните стъпки при разработка са: планиране, анализ, дизайн, имплементиране и поддръжка.



ОЩЕ

- **Идея** - анализиране на проблема
- **Предпроектно проучване** - дали реализацията е възможна и оправдана. Може да включва проучване на пазара
- Събиране на **изискванията за приложението** –резултатът е документ т.нар. Software Requirement Specification
- Изработване на план или **дизайн на приложението / архитектура** (изп. се блок-схеми, псевдокод и др.)
- **Писане на програмен код** (имплементация)
- **Тестване** на софтуера
- **Внедряване** при клиента
- **Поддръжка, отстраняване на бъгове и повишаване на сигурността**

Различните методологии(подходи) при разработката на софтуер (**waterfall, rapid application, agile и DevOps**) ги използват(частично или изцяло) в различна последователност или отделят повече или по-малко време на някои от тях. Детайлите в документацията на всеки етап също може да варират.

ПРОГРАМИРАНЕ - ДЕЙНОСТИ

Дейности, които касаят пряко процесът на програмиране:

- ✓ **написване** на програмата в текстов вид (кодиране)
- ✓ **превеждане** на текстовия код на програмата в изпълним код (транслация)
- ✓ **изпълнение** на програмата със съответните входни данни (изпълнение)
- ✓ **проверка за грешки** в програмата и тяхното отстраняване (тестване и настройка)
 - **разглеждане на резултатите от изпълнението** на програмата, за да се разбере какво е нейното действие
 - **изпълнение** на програмата с различни входни данни, за да се открият грешките (тестване)
 - **отстраняване** на грешките (настройка)
- ✓ **модифициране** на програмата, за да се адаптира към промените в света и нуждите на потребителите (поддръжка)
- ✓ **разширяване и допълване** на функционалните възможности на програмата, за да бъде тя по-полезна и удобна за потребителите (развитие)

ПРОГРАМНИ ГРЕШКИ

При комуникация на естествен език се допускат грешки – синтактични, граматически.

При създаването на една програма могат да се допуснат **три вида** грешки:

- **Синтактични грешки** – код в програмата, който е грешен от синтактична гледна точка;
- **Грешки по време на изпълнението на програмата** – програмни конструкции, които са синтактично правилни, но са безсмислени, напр. деление на 0, въвеждане на некоректни входни данни;
- **Логически грешки** – част от програмата, която е синтактично правилна, има смисъл, но е грешна от гледна точка на действията, които програмата трябва да изпълнява.

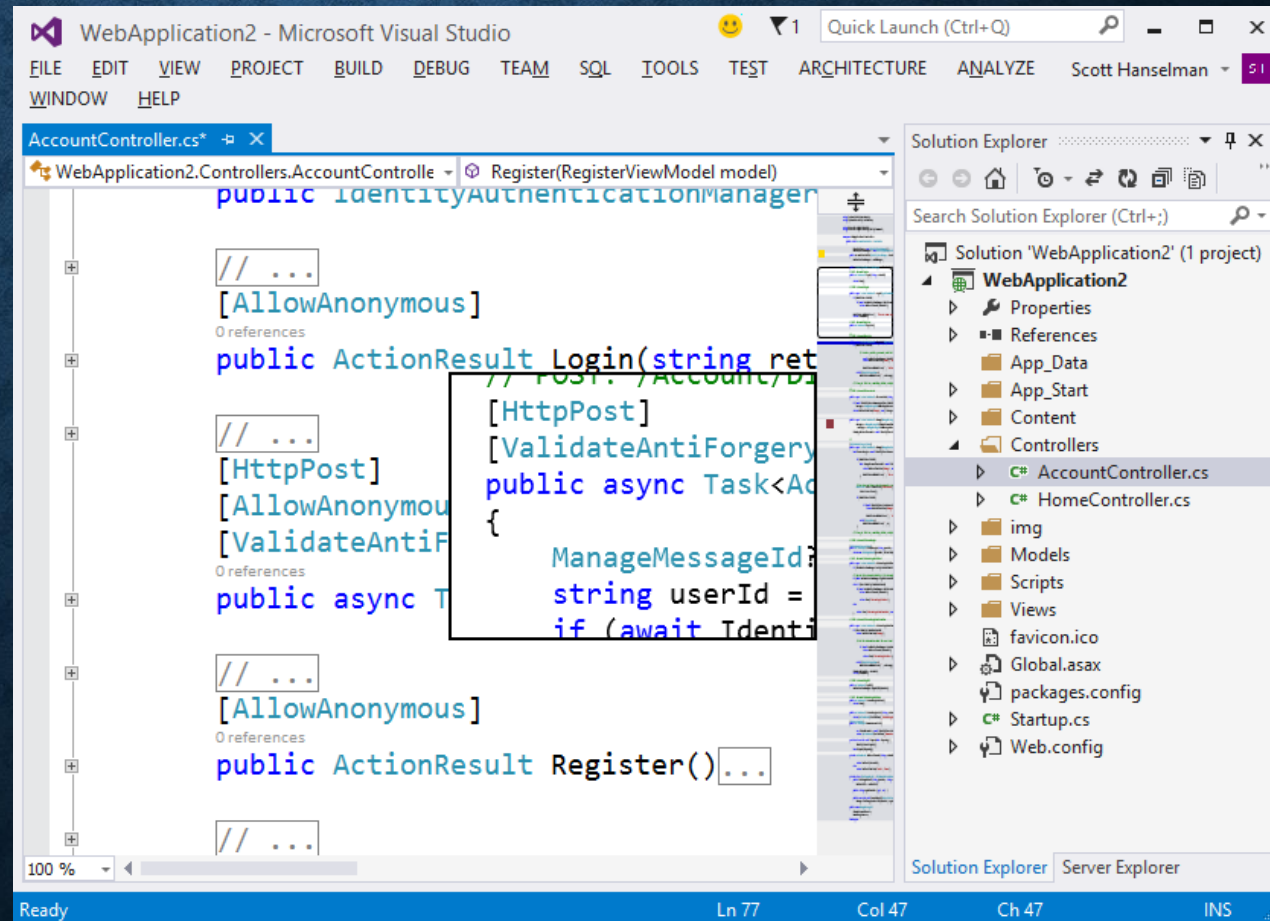
Примери

СРЕДИ ЗА ПРОГРАМИРАНЕ

- Среда за програмиране (IDE):
Съвкупност от програмни средства и инструменти за разработка на програми на един или няколко ЕП
- Предназначение:
 - цялостна поддръжка на програмните дейности
 - улеснява всички етапи от разработването на програмите
 - осигурява създаването на високо-качествени, ефективни и надеждни програми
- Могат да включват:
 - Редактори;
 - Дебъгери;
 - Компилатори;
 - Генератори на код;
 - Свързващи редактори и др.

СРЕДИ ЗА ПРОГРАМИРАНЕ: ПРОГРАМЕН РЕДАКТОР (PROGRAM EDITOR)

- Предназначение – написване (кодиране) на текста на програмата и създаване на файловете, от които се състои тя (напр. `Rational.cpp` в C++)
- Характеристики (контекстно-зависими):
 - улесняват кодирането, като предлагат избор между допустимите възможности при въвеждане на следващата конструкция според контекста, формиран до момента
 - подпомагат структурирането на текста и подобряват четливостта на програмата за човека
 - предупреждават за синтактични грешки, чрез форматиране на кода на базата на синтаксиса на използвания ЕП



СРЕДИ ЗА ПРОГРАМИРАНЕ: ТРАНСЛАТОРИ И СВЪРЗВАЩИ РЕДАКТОРИ

Транслатори

- Предназначение — превежда програмата, написана с помощта на редактора на език, разбираем за машината и я записва във файл (напр. `Calculator.obj` в средата на C++)
- Характеристики:
 - подпомага откриването и премахването на синтактични грешки
 - оптимизиращи транслатори
 - файлът, създаден от транслатора се нарича **обектен код** — предимно машинен код. Той притежава повечето от изграждащите блокове на програмния код освен системния код, който се намира във вградените библиотеки

Свързващи редактори (Linker)

- Свързва обектния код със съответния библиотечен код, т.е. свързването на изграждащите блокове в едно
- Създава изпълнимият файл (напр. `Calculator.exe` в C++)

СРЕДИ ЗА ПРОГРАМИРАНЕ: ЗАРЕЖДАЩА ПРОГРАМА И ДЕБЪГЕРИ

Зареждаща програма (Loader)

- Предназначение — зарежда изпълнима програма, създадена от компилатора в паметта на компютъра с цел тя да бъде изпълнена (при нейното стартиране)
- Характеристики:
 - тя е част от операционната система
 - след зареждането на програмата операционната система казва на процесора да започне изпълнението на последователността от инструкции от мястото в паметта, където сега се намира програмата

Дебъгери (Debugger)

- Предназначение — подпомага откриването и премахването на грешките по време на изпълнението на програмата и логическите грешки

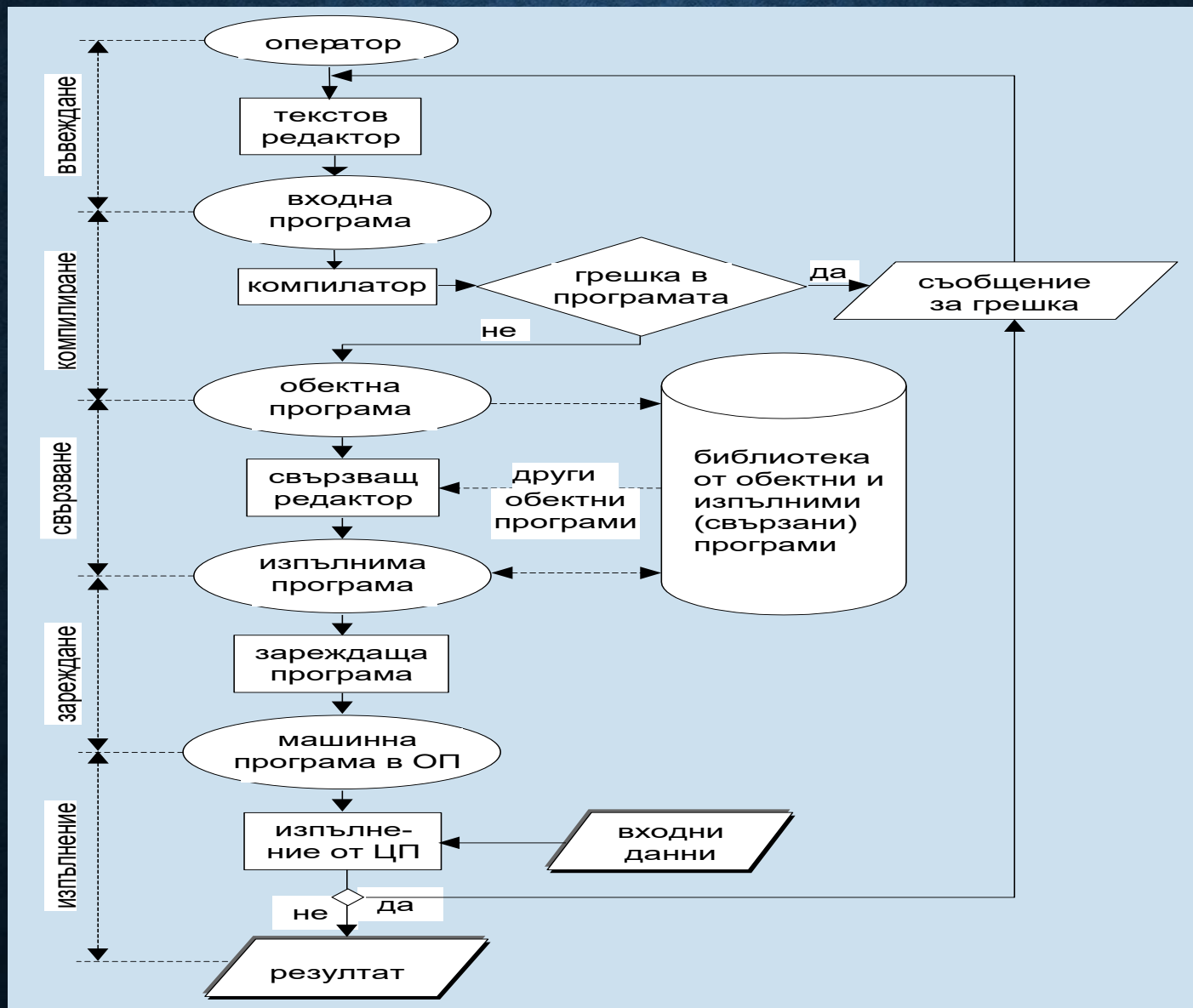
Самият процес се нарича **дебъгване** (*bug** — грешка)

Дебъгерът позволява т.нар. **трасиране** (*Tracing*) на програмата.

Трасирането включва следене на състоянието на паметта (напр. стойност на променлива) при изпълнението на програмните инструкции, така че цикълът от операции да може да се проследи и да се открият грешките

* **Терминът** *bug* е въведен от математичката Грейс Хопър, когато намира заклещена буболечка в изчислителната машина

СРЕДИ ЗА ПРОГРАМИРАНЕ: ЖИЗНЕН ЦИКЪЛ КОДИРАНЕ НА ПРОГРАМИ



ОСНОВЕН ПРИНЦИП: *ЕФЕКТИВНОСТ*

- по отношение на скорост
- по отношение на обем необходима памет
- ✓ скорост и леснота при писането на програма
- ✓ изразителност – използване на езици от високо ниво
- ✓ ефективност на транслиране – по-малки програми
- ✓ надеждност – използване на доказани ЕП и работни рамки
- ✓ лесна поддръжка - компоненти
- ✓ възможност за повторната употреба
- ✓ преносимост - мултиплатформеност

C++ ПРОГРАМИРАНЕ

- Езикът съчетава принципите на две програмни парадигми – процедурната и обектно-ориентираната
- създаден от Строуструп (Dr Bjarne Stroustrup) през 80-те години в AT&T Bell Labs
- последна версия от 2020 наречена C++20
- В настоящият курс не се акцентира на последните версии на езика, а се използват широко познати дори и да са по-стари езикови конструкции.

ОСНОВНИ ЛЕКСИКАЛНИ ЕЛЕМЕНТИ НА ПРОГРАМИТЕ

Лексикалните елементи (тоукън, *token* – знак, нещо значещо, символично) са основни градивни елементи на програмите, които са значещи за компилатора

- **име, идентификатор** (*identifier*) – последователност от символи, което служи за идентификация обикновено на програми (алгоритми), подпрограми (подалгоритми), елементи на данните и др.
- **ключова дума** (*keyword*) – предварително дефинирани имена, които имат специално предназначение при писане на програмите и не могат да бъдат използвани за други цели
- **константа** (*constant*)/**литерал** – стойност, която не се променя по време на изпълнение на програмата, а само се използва в процеса на изчисление
- **операция** (*operator*) – задава намирането на някаква стойност по дадени един, два или три операнда (съответно едно-, дву-, три-аргументна операция)
- **коментар** (*comment*) – текст, който се игнорира от компилатора, но е полезен за програмиста
- **препинателен знак** (*punctuator*) – ! % ^ & * () – + = { } | ~ [] \ ; ' : " < > ? , . / #

Лексикалните елементи на програмите се отделят с **разделители**, като празен интервал, край на ред, коментар и др.

ОСНОВНИ ЕЛЕМЕНТИ НА ЕП ОТ ГЛЕДНА ТОЧКА НА СИНТАКСИСА И СЕМАНТИКАТА

Най-общо ЕП включва средства за описание на два структурни елемента:

- а) **Деклариране и дефиниране на данните** – служи за определяне на компютърното представяне на входните и изходните данни и междинните резултати, на техния тип и структура, както и свързаните с тях изисквания за памет
- б) **Деклариране и дефиниране на обработката** – определя правилата за обработка на данните, как евентуално те се групират, както и последователността на тяхното прилагане чрез т.нар. **оператори** (команди, *statements*) на ЕП
 - **Декларацията** „казва“ на компилатора, че съответният програмен елемент съществува
 - **Дефиницията** определя точно кой код или данни декларираното име идентифицира
 - Името трябва да бъде декларирано преди да бъде използвано
 - Всеки елемент данни в програмата е или **константа** или **променлива (величини)**

ТИПИЧНА СТРУКТУРА НА ПРОГРАМА НА C++ – ПРИМЕР

- **Пример** (Програма, която извежда на екрана “Програмирането на C++ е лесно”):

```
/*  
    Първа програма на C++  
    за курса по Програмиране  
*/  
#include <iostream>  
using namespace std;  
main() //от тук започва изпълнението на програмата  
{  
    cout << "Програмирането на C++ е лесно.\n";  
}
```

- **Изпълнение на програмата:**

Програмирането на C++ е лесно.

ТИПИЧНА СТРУКТУРА НА ПРОГРАМА НА C++ – РЕЗЮМЕ ВЪРХУ ПРИМЕРА

- изпълнението на програмата започва от `main()`
- ключовите думи се пишат с малки букви
- всеки оператор завършва с `;`
- текстовите константи (низове) се заграждат в кавички (`"..."`)
- в C++ малките и големи букви се различават
- `\n` означава позициониране на курсора в началото на следващия нов ред
- `cout <<` може да се използва за извеждане на текст на екрана
- `{ }` определят началото и края на програмата (блок на програмата)
- `//` указва началото на коментар, който завършва в края на същия ред
- коментари с дължина повече от един ред се ограждат от `/* ... */`. Те не могат да се влагат

ТИПИЧНА СТРУКТУРА НА ПРОГРАМА НА C++

```
/* ЗАГЛАВНА СЕКЦИЯ                                */
/* Съдържа име, автор, номер версия                */

/* СЕКЦИЯ ЗА ВКЛЮЧВАНЕ НА ФАЙЛОВЕ                  */
/* Съдържа #include директиви                      */

/* СЕКЦИЯ ЗА ДЕКЛ. И ДЕФ. НА КОНСТАНТИ И ТИПОВЕ   */
/* Съдържа типове и директивата #define            */

/* СЕКЦИЯ ЗА ГЛОБАЛНИ ПРОМЕНЛИВИ                   */
/* Всички глобални променливи се декларират тук    */

/* СЕКЦИЯ ЗА ДЕКЛАРИРАНЕ И ДЕФИНИРАНЕ НА ФУНКЦИИ  */
/* Функции дефинирани от потребителя              */

/* main() СЕКЦИЯ (ГЛАВНА ФУНКЦИЯ)                  */

int main()
{
    //...
}
```

ДОПЪЛНИТЕЛНО СЪДЪРЖАНИЕ

- Не се включва в изпита
- По желание студентите могат да се запознаят допълнително и с възможностите за инструктиране на компилатора

ПРЕПРОЦЕСОРНИ ДИРЕКТИВИ

- Препроцесорът (preprocessor) е програма, която се стартира автоматично преди процеса на компилиране и извършва определени замествания в кода, дефинирани от програмистта, чрез препроцесорни директиви.
- Препроцесорните директиви винаги започват с '#', и са само един ред. Те се намират в началото на файла. Например директивата **#include <filename.h>** се заменя от препроцесора със съдържанието на указания файл.
- Друга препроцесорна директива е директивата **#define**. Резултатът от обработката на реда: **#define PI 3.14159** е заместване на всички срещания на думата **PI** в кода с **3.14159** (или това е дефиниране на константа).
- ```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

  
(или това е дефиниране на макрос функция)  

```
int x=5, y;
y= MAX(x, 2);
```
- Директива **#undef** „премахва“ преди деклариран идентификатор с **#define**



# ПРЕПРОЦЕСОРНИ ДИРЕКТИВИ

- Директивата **#include** се използва за включване на заглавни файлове (header files) към сорс файл. Заглавните файлове, наричани още хедъри, съдържат декларации на функции, константи и т.н. Тези файлове обикновено имат разширение 'h'. Когато препроцесорът срещне директива **#include**, той я заменя със съдържанието на указания хедър.

**#include** <myfile.h>      В този случай указаният файл се търси в някоя от include директориите на компилатора (например C:\Program Files\Microsoft Visual Studio 12.0\VC\INCLUDE).



# ПРЕПРОЦЕСОРНИ ДИРЕКТИВИ

- Директиви `#if`, `#elif`, `#else` и `#endif` се използват за условно изпълнение на компилацията на части от сорс файла.
- Ако изразът след `#if` има стойност различна от 0, то това което е след `#if` се подава за компилация.

```
#if TABLE_SIZE>200
#undef TABLE_SIZE
#define TABLE_SIZE 200
```

```
#elif TABLE_SIZE<50
#undef TABLE_SIZE
#define TABLE_SIZE 50
```

```
#else
#undef TABLE_SIZE
#define TABLE_SIZE 100
#endif
```

```
int table[TABLE_SIZE];
```



# ПРЕПРОЦЕСОРНИ ДИРЕКТИВИ

- Директива **#line** позволява да се управлява дефинираното по подразбиране (ред и файл) показване на грешки при компилация.
- Програмистът може по-разбираемо да показва грешките за свое или за друг удобство в проекти с множество сорс файлове, които споделят код един с друг.
- Но обикновено се използва в софтуер за генериране програмен код, за да даде точно местоположение на оригиналния сорс код
- Когато компилатора открие грешка в кода той показва текущия ред и файловото име.

- Синтаксис `#line number "filename",`

```
#include <assert.h>
#define FNAME "test.cpp"
void main()
{
 #line 777 FNAME
 assert(2+2 == 5); // макро функцията е за дебъгване
}
```